

【找数字】

题目描述

给一个 **二维数组** `nums`，对于每一个元素 `nums[i]`，找出距离最近的且值相等的元素，

输出横纵坐标差值的绝对值之和，如果没有等值元素，则输出-1。

例如：

输入数组 `nums` 为

```
0 3 5 4 2
2 5 7 8 3
2 5 4 2 4
```

输出为：

```
-1 4 2 3 3
1 1 -1 -1 4
1 1 2 3 2
```

- 对于 `nums[0][0] = 0`，不存在相等的值。
- 对于 `nums[0][1] = 3`，存在一个相等的值，最近的坐标为 `nums[1][4]`，最小距离为 4。
- 对于 `nums[0][2] = 5`，存在两个相等的值，最近的坐标为 `nums[1][1]`，故最小距离为 2。
- ...
- 对于 `nums[1][1] = 5`，存在两个相等的值，最近的坐标为 `nums[2][1]`，故最小距离为 1。
- ...

输入描述

输入第一行为二维数组的行

输入第二行为二维数组的列

输入的数字以空格隔开。

输出描述

数组形式返回所有 坐标值 。

备注

- 针对数组 nums[i][j]，满足 $0 < i \leq 100$ ， $0 < j \leq 100$
- 对于每个数字，最多存在 100 个与其相等的数字

用例

输入	3 5 0 3 5 4 2 2 5 7 8 3 2 5 4 2 4
输出	[[-1, 4, 2, 3, 3], [1, 1, -1, -1, 4], [1, 1, 2, 3, 2]]
说明	无

题目解析

我的解题思路如下：

首先遍历输入的矩阵，将相同数字的位置整理到一起。

然后再遍历一次输入的矩阵，找到和遍历元素相同的其他数字（通过上一步统计结果快速找到），求距离，并保留最小的距离。

上面逻辑的算法复杂度为 $O(n*m*k)$ ，其中 n 是输入矩阵行， m 是输入矩阵列， k 是每组相同数字的个数，可能会达到 $O(N^3)$ 的时间复杂度。

有一个优化点就是，遍历元素 A 找其他相同数字 B 时计算的距离可以缓存，这样的话，当遍历元素B时找其他相同数字A时，就可以从缓存中读取已经计算好的距离了，而不是重新计算。但是这样会浪费较多空间。

实际机试时，可以看用例数量级来决定是否要做上面这个优化。如果数量级很小，则无需上面优化。如果数量级较大，则有优化的必要。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const n = parseInt(await readline());
7      const m = parseInt(await readline());
8
9      const matrix = [];
10     for (let i = 0; i < n; i++) {
11         matrix.push((await readline()).split(" ").map(Number));
12     }
13
14     // 统计输入矩阵中，相同数字的位置
15     const nums = {};
16
17     for (let i = 0; i < n; i++) {
18         for (let j = 0; j < m; j++) {
19             const num = matrix[i][j];
20             nums[num] ? nums[num].push([i, j]) : (nums[num] = [[i, j]]);
21         }
22     }
```

运行

```

23 |
24 | // 遍历矩阵每一个元素，和其他相同数字的位置求距离，取最小距离
25 | for (let i = 0; i < n; i++) {
26 |     for (let j = 0; j < m; j++) {
27 |         const num = matrix[i][j];
28 |         let minDis = Infinity;
29 |
30 |         for (let [i1, j1] of nums[num]) {
31 |             if (i1 !== i || j1 !== j) {
32 |                 minDis = Math.min(minDis, Math.abs(i1 - i) + Math.abs(j1 - j));
33 |             }
34 |         }
35 |
36 |         matrix[i][j] = minDis === Infinity ? -1 : minDis;
37 |     }
38 | }
39 |
40 | console.log(JSON.stringify(matrix).replace(/,/g, ", "));
41 | })();

```

Java算法源码

```

1 | import java.util.ArrayList;
2 | import java.util.Arrays;
3 | import java.util.HashMap;
4 | import java.util.Scanner;
5 |
6 | public class Main {
7 |     public static void main(String[] args) {
8 |         Scanner sc = new Scanner(System.in);
9 |
10 |         int n = sc.nextInt();
11 |         int m = sc.nextInt();
12 |

```

```
13 // 统计输入矩阵中，相同数字的位置
14 |                                     HashMap<Integer, ArrayList<int[]>> nums = new HashMap<>();
```

```
15
16 int[][] matrix = new int[n][m];
17 for (int i = 0; i < n; i++) {
18     for (int j = 0; j < m; j++) {
19         int num = sc.nextInt();
20
21         matrix[i][j] = num;
22         nums.putIfAbsent(num, new ArrayList<>());
23         nums.get(num).add(new int[]{i, j});
24     }
25 }
```

```
26
27 // 遍历矩阵每一个元素，和其他相同数字的位置求距离，取最小距离
```

```
28 for (int i = 0; i < n; i++) {
29     for (int j = 0; j < m; j++) {
30         int num = matrix[i][j];
31         int minDis = Integer.MAX_VALUE;
32
33         for (int[] pos : nums.get(num)) {
34             int i1 = pos[0];
35             int j1 = pos[1];
36
37             if (i1 != i || j1 != j) {
38                 minDis = Math.min(minDis, Math.abs(i1 - i) + Math.abs(j1 - j));
39             }
40         }
41
42         matrix[i][j] = minDis == Integer.MAX_VALUE ? -1 : minDis;
43     }
44 }
45
46 System.out.println(Arrays.toString(Arrays.stream(matrix).map(Arrays::toString).toArray(String[]::new)));
47 }
48 }
```

Python算法源码

```
1 import sys
2
3
4 def solution():
5     # 输入获取
6     n = int(input())
7     m = int(input())
8     matrix = [list(map(int, input().split())) for _ in range(n)]
9
10    # 统计输入矩阵中，相同数字的位置
11    nums = {}
12
13    for i in range(n):
14        for j in range(m):
15            num = matrix[i][j]
16            if nums.get(num) is None:
17                nums[num] = [[i, j]]
18            else:
19                nums[num].append([i, j])
20
21    # 遍历矩阵每一个元素，和其他相同数字的位置求距离，取最小距离
22    for i in range(n):
23        for j in range(m):
24            num = matrix[i][j]
25            minDis = sys.maxsize
26
27            for i1, j1 in nums[num]:
28                if i1 != i or j1 != j:
29                    minDis = min(minDis, abs(i1 - i) + abs(j1 - j))
30
31            matrix[i][j] = -1 if minDis == sys.maxsize else minDis
32
33    return matrix
34
35
```

36 | # 输出打印

37 | `print(solution())`

C算法源码

C语言的Map太难手撕了，这里直接暴力了，经过测试也不会超时。

```
1  #include <stdio.h>
2  #include <math.h>
3  #include <limits.h>
4
5  int main() {
6      int n, m;
7      scanf("%d %d", &n, &m);
8
9      int matrix[n][m];
10     for (int i = 0; i < n; i++) {
11         for (int j = 0; j < m; j++) {
12             scanf("%d", &matrix[i][j]);
13         }
14     }
15
16     printf("[");
17     for (int i = 0; i < n; i++) {
18         printf("[");
19         for (int j = 0; j < m; j++) {
20             int minDis = INT_MAX;
21
22             for (int i1 = 0; i1 < n; i1++) {
23                 for (int j1 = 0; j1 < m; j1++) {
24                     if ((i1 != i || j1 != j) && matrix[i][j] == matrix[i1][j1]) {
25                         minDis = (int) fmin(minDis, abs(i1 - i) + abs(j1 - j));
26                     }
27                 }
28             }
29         }
30     }
31     printf("]");
32 }
```

```

29 |
30 |         printf("%d", minDis == INT_MAX ? -1 : minDis);
31 |
32 |         if (j < m - 1) {
33 |             printf(", ");
34 |         }
35 |     }
36 |     printf("]");
37 |
38 |     if (i < n - 1) {
39 |         printf(", ");
40 |     }
41 | }
42 | printf("]");
43 |
44 | return 0;
45 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int main() {
6 |     int n, m;
7 |     cin >> n >> m;
8 |
9 |     // 统计输入矩阵中，相同数字的位置
10 |    map<int, vector<int>>> nums;
11 |
12 |    vector<vector<int>>> matrix(n, vector<int>(m));
13 |    for (int i = 0; i < n; i++) {
14 |        for (int j = 0; j < m; j++) {
15 |            int num;

```



```

16         cin >> num;17 |
18         matrix[i][j] = num;
19         nums[num].emplace_back(i * m + j); // 二维坐标一维化
20     }
21 }
22
23 // 遍历矩阵每一个元素，和其他相同数字的位置求距离，取最小距离
24 cout << "[";
25 for (int i = 0; i < n; i++) {
26     cout << "[";
27     for (int j = 0; j < m; j++) {
28         int num = matrix[i][j];
29         int minDis = INT_MAX;
30
31         for (const auto &pos: nums[num]) {
32             int i1 = pos / m;
33             int j1 = pos % m;
34
35             if (i != i1 || j != j1) {
36                 minDis = min(minDis, abs(i1 - i) + abs(j1 - j));
37             }
38         }
39
40         cout << ((minDis == INT_MAX) ? -1 : minDis);
41
42         if (j < m - 1) {
43             cout << ", ";
44         }
45     }
46     cout << "];
47
48     if (i < n - 1) {
49         cout << ", ";
50     }
51 }
52 cout << "];
53

```

```
54 |     return 0;  
55 | }
```